Pong

Passed by Group 7:

Derilo, Ashley Shane

Francisco, Florenz Jay

Gumia, Samuel Josh

Gutierrez, Raven Jay

Pong is a game of 2 players, it is not an original game from us but we can add a few twists here and there. It is a game where 2 players have to protect their goal from the ball at the same time attacking each other. A player with the most points by the end of the game wins.

How to play?

Player 1:

W – move up

S – move down

Player 2:

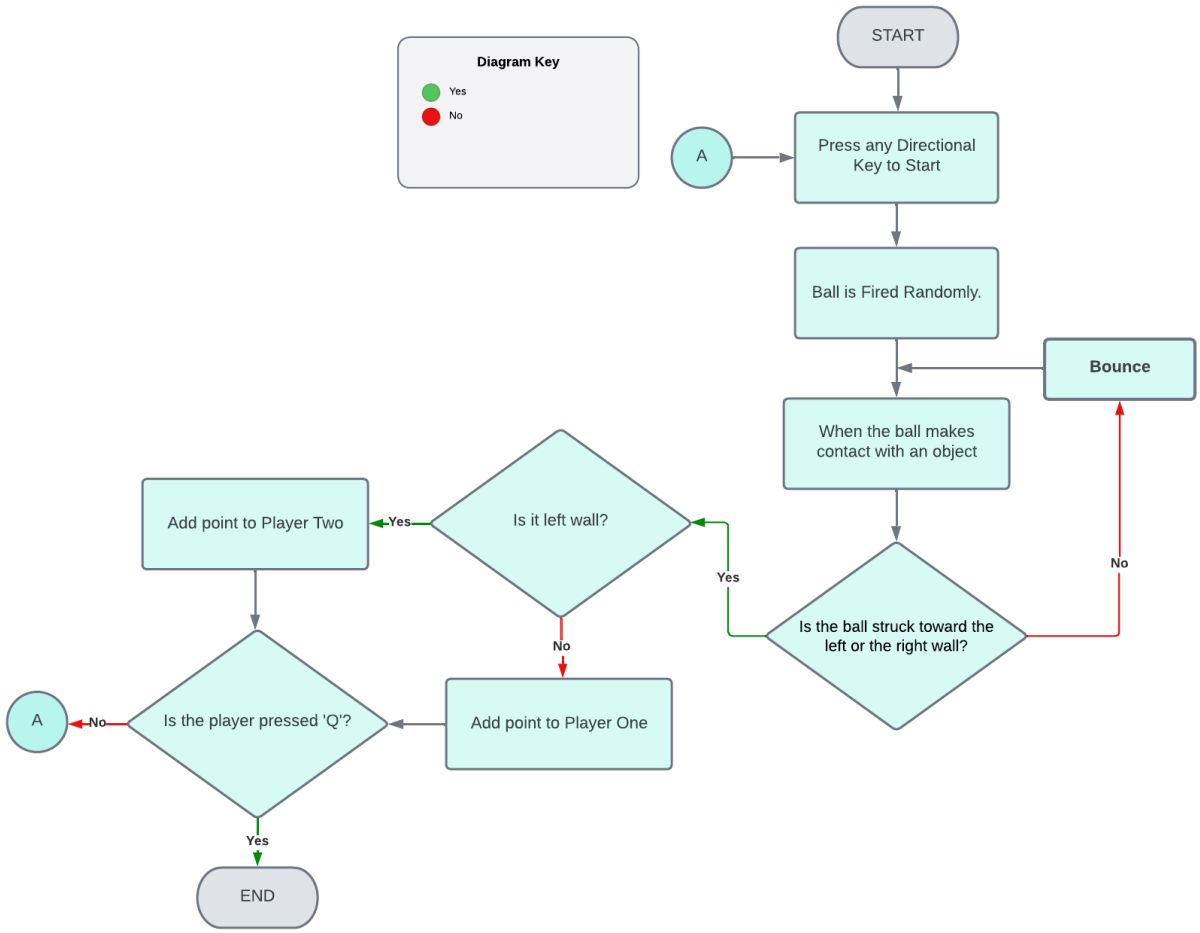
I - move up

K – move down

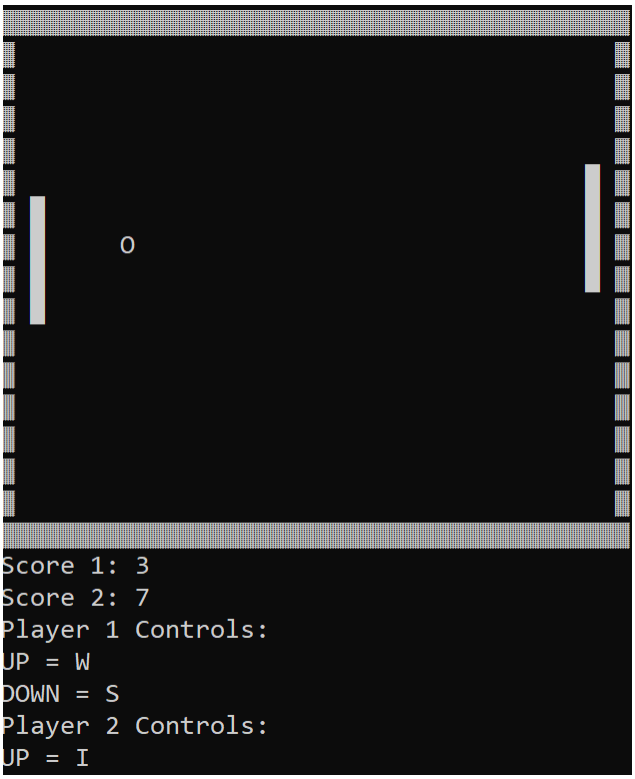
Global:

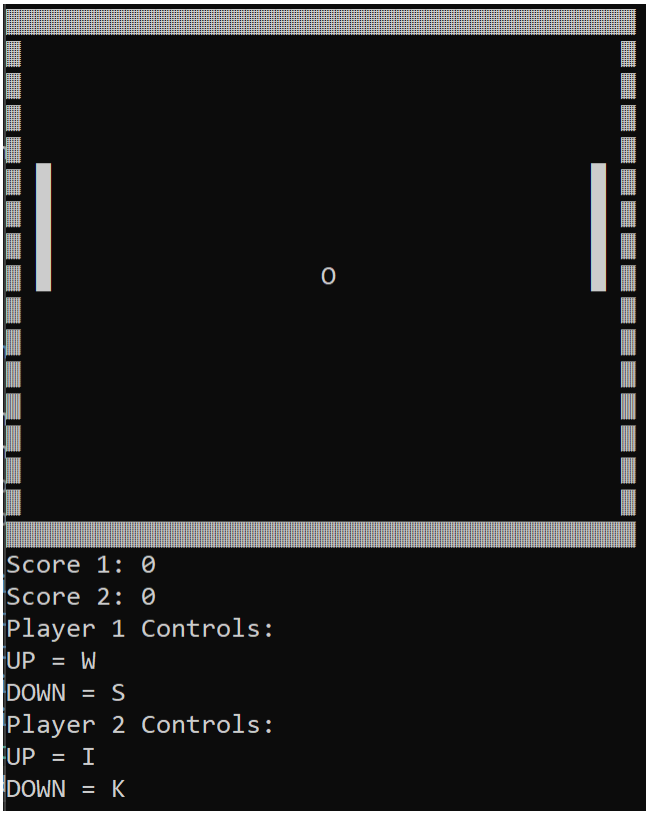
Q – exit the game

Flowchart



Sample Output:





Source Code:

```
#include <iostream>
#include <time.h>
#include <conio.h>
using namespace std;

enum eDir { STOP = 0, LEFT = 1, UPLEFT = 2, DOWNLEFT = 3, RIGHT = 4, UPRIGHT = 5,
DOWNRIGHT = 6 };

class cBall
{
private:
    int x, y;
    int originalX, originalY;
    eDir direction;
public:
    cBall(int posX, int posY)
    {
        originalX = posX;
        originalY = posY;
        x = posX; y = posY;
        direction = STOP;
    }
    void Reset()
    {
        x = originalX; y = originalY;
        direction = STOP;
    }
    void changeDirection(eDir d)
    {
        direction = d;
    }
    void randomDirection()
    {
        direction = (eDir)((rand() % 6) + 1);
    }
    inline int getX() { return x; }
    inline int getY() { return y; }
    inline eDir getDirection() { return direction; }
    void Move()
    {
        switch (direction)
        {
        case STOP:
            break;
        case LEFT:
            x--;
            break;
        case RIGHT:
            x++;
            break;
        case UPLEFT:
            x--; y--;
            break;
        case DOWNLEFT:
            x--; y++;
            break;
```

```
        case UPRIGHT:
            x++; y--;
            break;
        case DOWNRIGHT:
            x++; y++;
            break;
        default:
            break;
        }
    }
    friend ostream& operator<<(ostream& o, cBall c)
    {
        o << "Ball [" << c.x << "," << c.y << "][" << c.direction << "]";
        return o;
    }
};
class cPlayer
{
private:
    int x, y;
    int originalX{}, originalY{};
public:
    cPlayer()
    {
        x = y = 0;
    }
    cPlayer(int posX, int posY) : cPlayer()
    {
        originalX = posX;
        originalY = posY;
        x = posX;
        y = posY;
    }
    inline void Reset() { x = originalX; y = originalY; }
    inline int getX() { return x; }
    inline int getY() { return y; }
    inline void moveUp() { y--; }
    inline void moveDown() { y++; }
    friend ostream& operator<<(ostream& o, cPlayer c)
    {
        o << "Paddle [" << c.x << "," << c.y << "]";
        return o;
    }
};
class cGameManger
{
private:
    int width, height;
    int score1, score2;
    char up1, down1, up2, down2;
    bool quit;
    cBall* ball;
    cPlayer* player1;
    cPlayer* player2;
public:
    cGameManger(int w, int h)
    {
        srand(time(NULL));
```

```
    quit = false;
    up1 = 'w'; up2 = 'i';
    down1 = 's'; down2 = 'k';
    score1 = score2 = 0;
    width = w; height = h;
    ball = new cBall(w / 2, h / 2);
    player1 = new cPlayer(1, h / 2 - 3);
    player2 = new cPlayer(w - 2, h / 2 - 3);
}
~cGameManger()
{
    delete ball, player1, player2;
}
void ScoreUp(cPlayer* player)
{
    if (player == player1)
        score1++;
    else if (player == player2)
        score2++;

    ball->Reset();
    player1->Reset();
    player2->Reset();
}
void Draw()
{
    system("cls");
    for (int i = 0; i < width + 2; i++)
        cout << "\xB2";
    cout << endl;

    for (int i = 0; i < height; i++)
    {
        for (int j = 0; j < width; j++)
        {
            int ballx = ball->getX();
            int bally = ball->getY();
            int player1x = player1->getX();
            int player2x = player2->getX();
            int player1y = player1->getY();
            int player2y = player2->getY();

            if (j == 0)
                cout << "\xB2";

            if (ballx == j && bally == i)
                cout << "O";
            else if (player1x == j && player1y == i)
                cout << "\xDB";
            else if (player2x == j && player2y == i)
                cout << "\xDB";

            else if (player1x == j && player1y + 1 == i)
                cout << "\xDB";
            else if (player1x == j && player1y + 2 == i)
                cout << "\xDB";
            else if (player1x == j && player1y + 3 == i)
                cout << "\xDB";
```

```
                else if (player2x == j && player2y + 1 == i)
                    cout << "\xDB";
                else if (player2x == j && player2y + 2 == i)
                    cout << "\xDB";
                else if (player2x == j && player2y + 3 == i)
                    cout << "\xDB";
                else
                    cout << " ";

                if (j == width - 1)
                    cout << "\xB2";
            }
            cout << endl;
        }

        for (int i = 0; i < width + 2; i++)
            cout << "\xB2";
        cout << endl;

        cout << "Score 1: " << score1 << endl << "Score 2: " << score2 << endl;
        cout << "Player 1 Controls: " << endl << "UP = W" << endl << "DOWN = S" <<
endl;
        cout << "Player 2 Controls: " << endl << "UP = I" << endl << "DOWN = K" <<
endl;
    }
    void Input()
    {
        ball->Move();

        int ballx = ball->getX();
        int bally = ball->getY();
        int player1x = player1->getX();
        int player2x = player2->getX();
        int player1y = player1->getY();
        int player2y = player2->getY();

        if (_kbhit())
        {
            char current = _getch();
            if (current == up1)
                if (player1y > 0)
                    player1->moveUp();
            if (current == up2)
                if (player2y > 0)
                    player2->moveUp();
            if (current == down1)
                if (player1y + 4 < height)
                    player1->moveDown();
            if (current == down2)
                if (player2y + 4 < height)
                    player2->moveDown();

            if (ball->getDirection() == STOP)
                ball->randomDirection();

            if (current == 'q')
                quit = true;
```

```
        }
    }
    void Logic()
    {
        int ballx = ball->getX();
        int bally = ball->getY();
        int player1x = player1->getX();
        int player2x = player2->getX();
        int player1y = player1->getY();
        int player2y = player2->getY();

        for (int i = 0; i < 4; i++)
            if (ballx == player1x + 1)
                if (bally == player1y + i)
                    ball->changeDirection((eDir)((rand() % 3) + 4));

        for (int i = 0; i < 4; i++)
            if (ballx == player2x - 1)
                if (bally == player2y + i)
                    ball->changeDirection((eDir)((rand() % 3) + 1));

        if (bally == height - 1)
            ball->changeDirection(ball->getDirection() == DOWNRIGHT ? UPRIGHT :
UPLEFT);
        if (bally == 0)
            ball->changeDirection(ball->getDirection() == UPRIGHT ? DOWNRIGHT :
DOWNLEFT);
        if (ballx == width - 1)
            ScoreUp(player1);
        if (ballx == 0)
            ScoreUp(player2);
    }
    void Run()
    {
        while (!quit)
        {
            Draw();
            Input();
            Logic();
        }
    }
};
int main()
{
    cGameManger c(40, 15);
    c.Run();
    return 0;
}
```